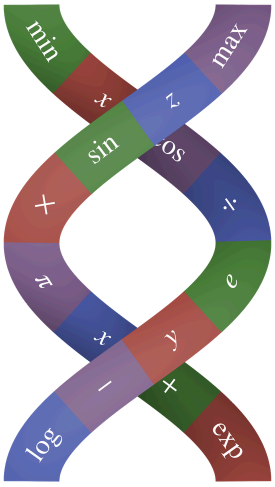
PySR




# Discover Mathematical Laws from Data

High-performance symbolic regression in Python and Julia

Get Started  API Reference 📚  View on GitHub

🔬

### Interpretable By Design

Discovers interpretable mathematical expressions instead of black-box models.

🚀

### Production Ready

Years of development have produced mature, highly optimized parallel evolutionary algorithms.

🔧

### Extremely Customizable

Customize everything: operators, loss functions, complexity, input types, optimizer, and more.

🐍

### Familiar Python API

A scikit-learn interface, with results exported to SymPy, PyTorch, or JAX.

---

If you find PySR useful, please cite the paper arXiv:2305.01582. If you've finished a project with PySR, please submit a PR to showcase your work on the research showcase page!

---

PySR is an open-source tool for *Symbolic Regression*: a machine learning task where the goal is to find an interpretable symbolic expression that optimizes some objective.

Over a period of several years, PySR has been engineered from the ground up to be (1) as high-performance as possible, (2) as configurable as possible, and (3) easy to use. PySR is developed alongside the Julia library SymbolicRegression.jl, which forms the powerful search engine of PySR. The details of these algorithms are described in the PySR paper.

Symbolic regression works best on low-dimensional datasets, but one can also extend these approaches to higher-dimensional spaces by using "*Symbolic Distillation*" of Neural Networks, as explained in 2006.11287, where we apply it to N-body problems. Here, one essentially uses symbolic regression to convert a neural net to an analytic equation. Thus, these tools simultaneously present an explicit and powerful way to interpret deep neural networks.

You can install PySR with pip or conda:

```bash
pip install pysr
```

```bash
conda install -c conda-forge pysr
```

Julia dependencies will be installed at first import.

Install the PySR skill to provide your coding agents guidance using PySR as part of a larger agentic loop:

```bash
npx skills add astroautomata/PySR
```

Add `--global` to install the skill globally.

▶

▶

▶

You might wish to try the interactive tutorial here, which uses the notebook in `examples/pysr_demo.ipynb` .

In practice, I highly recommend using IPython rather than Jupyter, as the printing is much nicer. Below is a quick demo here which you can paste into a Python runtime. First, let's import numpy to generate some test data:

```python
import numpy as np

X = 2 * np.random.randn(100, 5)
y = 2.5382 * np.cos(X[:, 3]) + X[:, 0] ** 2 - 0.5
```

We have created a dataset with 100 datapoints, with 5 features each. The relation we wish to model is $2.5382\cos(x_3) + x_0^2 - 0.5$.

Now, let's create a PySR model and train it. PySR's main interface is in the style of scikit-learn:

```python
from pysr import PySRRegressor

model = PySRRegressor(
    maxsize=20,
    niterations=40,  # < Increase me for better results
    binary_operators=["+", "*"],
    unary_operators=[
        "cos",
        "exp",
        "sin",
        "inv(x) = 1/x",
        # ^ Custom operator (julia syntax)
    ],
    extra_sympy_mappings={"inv": lambda x: 1 / x},
    # ^ Define operator for SymPy as well
    elementwise_loss="loss(prediction, target) = (prediction - target)^2",
    # ^ Custom loss function (julia syntax)
)
```

This will set up the model for 40 iterations of the search code, which contains hundreds of thousands of mutations and equation evaluations.

Let's train this model on our dataset:

```python
model.fit(X, y)
```

Internally, this launches a Julia process which will do a multithreaded search for equations to fit the dataset.

Equations will be printed during training, and once you are satisfied, you may quit early by hitting 'q' and then <enter>.

After the model has been fit, you can run `model.predict(X)` to see the predictions on a given dataset using the automatically-selected expression, or, for example, `model.predict(X, 3)` to see the predictions of the 3rd equation.

You may run:

```python
print(model)
```

to print the learned equations:

```python
PySRRegressor.equations_ = [
       pick     score                                           equation
    0        0.000000                                          4.4324794
    1        1.255691                                          (x0 * x0)
    2        0.011629                          ((x0 * x0) + -0.28087974)
    3        0.897855                              ((x0 * x0) + cos(x3))
    4        0.857018                    ((x0 * x0) + (cos(x3) * 2.4566472))
    5  >>>>       inf  (((cos(x3) + -0.19699033) * 2.5382123) + (x0 *...
]
```

This arrow in the `pick` column indicates which equation is currently selected by your `model_selection` strategy for prediction. (You may change `model_selection` after `.fit(X, y)` as well.)

`model.equations_` is a pandas DataFrame containing all equations, including callable format ( `lambda_format` ), SymPy format ( `sympy_format` - which you can also get

with `model.sympy()`), and even JAX and PyTorch format (both of which are differentiable - which you can get with `model.jax()` and `model.pytorch()`).

Note that `PySRRegressor` stores the state of the last search, and will restart from where you left off the next time you call `.fit()`, assuming you have set `warm_start=True`. This will cause problems if significant changes are made to the search parameters (like changing the operators). You can run `model.reset()` to reset the state.

You will notice that PySR will save two files: `hall_of_fame...csv` and `hall_of_fame...pkl`. The csv file is a list of equations and their losses, and the pkl file is a saved state of the model. You may load the model from the `pkl` file with:

python
```
model = PySRRegressor.from_file("hall_of_fame.2022-08-10_100832.281.pkl")
```

There are several other useful features such as denoising (e.g., `denoise=True`), feature selection (e.g., `select_k_features=3`). For examples of these and other features, see the examples page. For a detailed look at more options, see the options page. You can also see the full API at this page. There are also tips for tuning PySR on this page.

The following code makes use of as many PySR features as possible. Note that is just a demonstration of features and you should not use this example as-is. For details on what each parameter does, check out the API page.

python
```
model = PySRRegressor(
    populations=8,
    # ^ Assuming we have 4 cores, this means 2 populations per core, so on
    population_size=50,
    # ^ Slightly larger populations, for greater diversity.
    ncycles_per_iteration=500,
    # ^ Generations between migrations.
    niterations=10000000,  # Run forever
    early_stop_condition=(
```

```
    "stop_if(loss, complexity) = loss < 1e-6 && complexity < 10"
    # Stop early if we find a good and simple equation
),
timeout_in_seconds=60 * 60 * 24,
# ^ Alternatively, stop after 24 hours have passed.
maxsize=50,
# ^ Allow greater complexity.
maxdepth=10,
# ^ But, avoid deep nesting.
binary_operators=["*", "+", "-", "/"],
unary_operators=["square", "cube", "exp", "cos2(x)=cos(x)^2"],
constraints={
    "/": (-1, 9),
    "square": 9,
    "cube": 9,
    "exp": 9,
},
# ^ Limit the complexity within each argument.
# "inv": (-1, 9) states that the numerator has no constraint,
# but the denominator has a max complexity of 9.
# "exp": 9 simply states that `exp` can only have
# an expression of complexity 9 as input.
nested_constraints={
    "square": {"square": 1, "cube": 1, "exp": 0},
    "cube": {"square": 1, "cube": 1, "exp": 0},
    "exp": {"square": 1, "cube": 1, "exp": 0},
},
# ^ Nesting constraints on operators. For example,
# "square(exp(x))" is not allowed, since "square": {"exp": 0}.
complexity_of_operators={"/": 2, "exp": 3},
# ^ Custom complexity of particular operators.
complexity_of_constants=2,
# ^ Punish constants more than variables
select_k_features=4,
# ^ Train on only the 4 most important features
progress=True,
# ^ Can set to false if printing to a file.
weight_randomize=0.1,
# ^ Randomize the tree much more frequently
cluster_manager=None,
# ^ Can be set to, e.g., "slurm", to run a slurm
```

```
    # cluster. Just launch one script from the head node.
    precision=64,
    # ^ Higher precision calculations.
    warm_start=True,
    # ^ Start from where left off.
    turbo=True,
    # ^ Faster evaluation (experimental)
    extra_sympy_mappings={"cos2": lambda x: sympy.cos(x)**2},
    # extra_torch_mappings={sympy.cos: torch.cos},
    # ^ Not needed as cos already defined, but this
    # is how you define custom torch operators.
    # extra_jax_mappings={sympy.cos: "jnp.cos"},
    # ^ For JAX, one passes a string.
)
```

You can also test out PySR in Docker, without installing it locally, by running the following command in the root directory of this repo:

```bash
docker build -t pysr .
```

This builds an image called `pysr` for your system's architecture, which also contains IPython. You can select a specific version of Python and Julia with:

```bash
docker build -t pysr --build-arg JLVERSION=1.10.0 --build-arg PYVERSION=3.
```

You can then run with this dockerfile using:

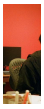
```bash
docker run -it --rm -v "$PWD:/data" pysr ipython
```

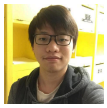
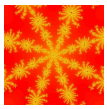
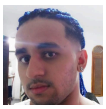
which will link the current directory to the container's `/data` directory and then launch ipython.

If you have issues building for your system's architecture, you can emulate another architecture by including `--platform linux/amd64`, before the `build` and `run` commands.

✨

We are eager to welcome new contributors! Check out our contributors guide for tips 🚀. If you have an idea for a new feature, don't hesitate to share it on the issues or discussions page.

<table>
<tr>
<td>  <br>Adil</td>
<td>  <br>Christopher Rowley</td>
<td>  <br>Mark Kittisopikul</td>
<td>  <br>T_Coxon</td>
<td>  <br>Dhananjay Ashok</td>
<td>  <br>Johan Blåbäck</td>
<td>  <br>JuliusM</td>
</tr>
<tr>
<td>  <br>Kaze Wong</td>
<td>  <br>Christopher Rackauckas</td>
<td>  <br>Patrick Kidger</td>
<td>  <br>Okon Samuel</td>
<td>  <br>William Booth-Clibborn</td>
<td>  <br>Aya Ghaleb</td>
<td>  <br>gc</td>
</tr>
<tr>
<td>  <br>Atharva Sehgal</td>
<td>  <br>wkharold</td>
<td>  <br>William Moses</td>
<td>  <br>Ardeleanu Cristian</td>
<td>  <br>gm89uk</td>
<td>  <br>Pablo Lemos</td>
<td>  <br>Jerr</td>
</tr>
<tr>
<td>  <br>Johann Brehmer</td>
<td>  <br>Marius Millea</td>
<td>  <br>Coba</td>
<td>  <br>foxtran</td>
<td>  <br>Shah Mahdi Hasan</td>
<td>  <br>Dilum Aluthge</td>
<td>  <br>Seb Mic Câm</td>
</tr>
</table>

| | | | | | | |
|---|---|---|---|---|---|---|
|  BrotherHa |  William Thompson |  Tom Jelen |  Miguel Crispim Romao |  Andy Dienes |  Harsh Singh |  Pi Mon |
|  William Booth-Clibborn |  Raúl Peralta Lozada |  Roy Hvaara |  Vishal Jain |  spaette |  Yi-Xin Liu |  spi |
|  Jay Wadekar |  Anthony Blaom, PhD |  Jgmedina95 |  Michael Abbott |  Oscar Smith |  Eric Hanson |  Henriqu |
|  Rik Huijzer |  Hongyu Wang |  Zehao Jin |  Tanner Mengel |  Arthur Grundner |  sjwetzel |  Sa Mah |
|  Ilya Orson |  Ho Fung Tsoi |  LionessOfCintra |  Manuel Morales |  Paulo Montero Camacho |  Writu Dasgupta |  Anubh |
|  Hao Guo |  Trailblazer |  Christos Pliakos |  Sousa Neto |  Leonardo Voltolini |  Daniel Eduardo |  Sam E |

| | | | | | | |
|---|---|---|---|---|---|---|
| | | | | | Conde Villatoro | |

Made with VitePress
© Copyright 2026.